Reviewer Pack — Projective Plane Incidence Bounds Nisan-Wigderson Seed Lengt…

Entry c9054923e964 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 04:40:02 UTC

Projective Plane Incidence Bounds Nisan-Wigderson Seed Length

Entry ID: `c9054923e964` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 04:40:02 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry (Projective Planes) **Field B** (complexity object): Nisan-Wigderson PRG Seed Length

Statement:

> For a CNF Φ with n variables whose clause incidence matrix is a projective plane of order q, the Nisan-Wigderson seed length for Φ is $\leq q^2 + q + 1$. Conversely, if Φ has seed length $> q^2 + q + 1$, its incidence matrix cannot be a projective plane of order q.

Rationale (proposer's reasoning):

> Projective planes exhibit optimal combinatorial expansion properties that could constrain pseudorandom generator seed requirements. The bound q^2+q+1 matches the number of lines in the plane, suggesting a direct geometric constraint on the PRG's pseudorandomness threshold.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `13a3271a1d21ee4c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from collections import defaultdict

def finite_field_add(a, b, q):
    return (a + b) % q

def finite_field_mul(a, b, q):
    return (a * b) % q

def finite_field_inv(a, q):
    for i in range(1, q):
        if finite_field_mul(a, i, q) == 1:
            return i
    raise ValueError("No inverse found")

def gaussian_elimination(matrix, n, q):
    for i in range(n):
        # Find pivot row
        pivot_row = i
        while pivot_row < n and matrix[pivot_row][i] == 0:
            pivot_row += 1
        if pivot_row == n:
            continue

        # Swap rows
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]

        # Eliminate below pivot
        for j in range(i + 1, n):
            factor = finite_field_mul(matrix[j][i], finite_field_inv(matrix[i][i], q), q)
            for k in range(n + 1):
                matrix[j][k] = finite_field_add(matrix[j][k], finite_field_mul(-factor, matrix[i][

def rank(matrix, n, q):
    gaussian_elimination(matrix, n, q)
    return sum(1 for row in matrix if any(x != 0 for x in row[:n]))

def generate_projective_plane(q):
```

```
    if q < 2:
        raise ValueError("q must be at least 2")

    points = [(i, j) for i in range(q) for j in range(q)]
    lines = []

    # Vertical lines
    for i in range(q):
        line = [(i, j) for j in range(q)]
        lines.append(line)

    # Horizontal lines
    for j in range(q):
        line = [(i, j) for i in range(q)]
        lines.append(line)

    # Diagonal lines
    for k in range(2 * q - 1):
        if k < q:
            line = [(i, (i + k) % q) for i in range(q)]
        else:
            line = [(q - 1 - (k - q), i) for i in range(q)]
        lines.append(line)

    return points, lines

def generate_cnf_from_plane(points, lines):
    n = len(points)
    m = len(lines)
    cnf = []

    # Each point must be on at least one line
    for i in range(n):
        clause = [j + 1 for j in range(m) if (i // q, i % q) in lines[j]]
        cnf.append(clause)

    return cnf

def compute_seed_length(cnf):
    n = len(cnf)
    m = sum(len(clause) for clause in cnf)
    seed_length = 0

    # Simulate iterative partitioning
    while m > 1:
        seed_length += 1
        m = math.ceil(m / 2)

    return seed_length

def run_trial(seed: int) -> dict:
    random.seed(seed)

    q_values = [2, 3]
    results = []
```

```
    for q in q_values:
        points, lines = generate_projective_plane(q)
        cnf = generate_cnf_from_plane(points, lines)

        if len(cnf) != q * (q + 1):
            return {
                "metric_name": "seed_length",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        seed_length = compute_seed_length(cnf)
        expected_bound = q**2 + q + 1

        results.append({
            "q": q,
            "seed_length": seed_length,
            "expected_bound": expected_bound
        })

    conjecture_holds = all(result["seed_length"] <= result["expected_bound"] for result in results
    counterexample = "" if conjecture_holds else f"q={results[0]['q']}, seed_length={results[0]['s

    return {
        "metric_name": "seed_length",
        "metric_value": sum(result["seed_length"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_seed_length = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_seed_length} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"q={results[0]['q']}, seed_length={results[0]['s
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54288a4f.py", line 148, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54288a4f.py", line 110, in run_trial
    cnf = generate_cnf_from_plane(points, lines)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_54288a4f.py", line 85, in generate_cnf_
    clause = [j + 1 for j in range(m) if (i // q, i % q) in lines[j]]
                                                 ^
NameError: name 'q' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined variable ‘q’ in the generation logic, preventing validation of the conjecture | next: Fix the code to properly define ‘q’ in the projective plane generation and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	52460
2	preregistration	ollama_remote	qwen3:8b	0	0	12823
3	novelty	ollama_remote	qwen3:8b	0	0	10071
4	novelty	ollama_remote	qwen3:8b	0	0	8100
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16805
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14943
7	judge	ollama_remote	qwen3:8b	0	0	17667

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 132870 ms total latency. Provider mix: {‘ollama_remote’: 7}

(full prompt+response transcripts available in `research/audit/c9054923e964.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c9054923e964.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c9054923e964.tar.gz` (if generated)